

DACON 数智大会 · 技术实践专场

基于Harness Engineering 一站式交付落地实践

路芳芳 | 中兴通讯 资深工程师



路芳芳

中兴通讯 · 资深工程师

· 2007 年加入中兴，深耕研发一线 19 年

· 前 10 年专注应用开发，5 年带领研发团队

· 近 4 年转型研发流程改进

· 2025 年主导"一站式提效"在多项目规模化落地，实现 30% 需求无人工干预交付

开场 · 三个数字

在分享方案之前，先看我们面对的三个数字

400+

每周新增的故障告警

每一个故障都意味着定界、指派、分析，一条看不到尽头的人工流水线。

1~4小时

处理一个故障的平均耗时

工程师被迫中断手头开发，切换进历史代码工程，上下文反复重建。

4次

一个需求的人工转译次数

BA → SE → Dev → QA，每过一次手，需求就失真一次，"交付物不符合预期"成为常态。

这三个数字背后，是同一件事：**传统研发模式的局部优化，边际收益已经很低了**

今天讲什么

40 分钟，回答四个问题

01

问题提出：传统研发提效为什么到顶了？

需求开发的三大核心问题，运维诊断的三重效率困境

02

解决思路：什么是"一站式交付"与 Harness Engineering？

从多角色接力到单节点全栈闭环；约束、反馈、验证、自我修正

03

实践情况：这套体系是怎么搭出来的？

两层工程底座、六大核心 Skill、三个真实案例

04

效果评价：数字说话，带来了什么改变？

月均 400 单无人工干预交付、效率提升 6 倍、代码采纳率 70%

01

问题提出

传统研发提效，为什么到顶了？

问题提出 · 需求开发

需求开发的三大核心问题

角色固化

串行研发模式下，角色边界固化导致"知识孤岛"，知识无法跨角色流动。

研发提效长期局限于局部节点的流转速度，而非全局产出

交付断层

需求在 BA → SE → Dev → QA 的流转中存在严重的感知衰减，信息传递层层失真。

多轮人工转译引发信息熵增，"交付物不符合预期"反复出现

响应滞后

固化的角色分工使得单一需求必须排队经过完整链路，需求排队等待处理。

无法实现极致的即时反馈与敏捷迭代

提效瓶颈： 多角色分工导致的感知衰减，异步协作产生的交付断层

问题提出 · 运维诊断

运维诊断的三重效率困境

人工处理，流程慢

故障定界、指派与分析重度依赖人工经验；人工驱动的模式无法匹配高频迭代节奏，直接制约版本交付效率。

工作切换，成本高

开发人员被迫中断当前任务，回溯历史需求与方案，承担极高的认知重构与业务环境重构成本。

资源竞争，冲突多

需求开发与运维诊断持续争抢人力与环境资源，临时"技术补丁"不断转化为长期技术债务。

影响传导链：恶性循环由此形成

故障发生 平均 400+/周 → 人工介入 平均耗时 1~4h → 开发中断 上下文丢失 → 效率下降 交付延期 → 恶性循环 问题累积

往深看一层：问题不在人，在模式

多角色接力，必然带来信息熵增；

人工驱动，必然存在吞吐瓶颈。

过去我们一直在优化"每一棒"跑得更快，

但如果换一种问法——能不能不要接力棒？

这正是"一站式交付"要回答的问题

02

解决思路

从"多角色接力"到"单节点全栈闭环"

一站式交付解决方案总体架构

应用场景

- 一站式需求开发 · 全栈模式变革
- 一站式运维诊断 · 熄灯工厂

入口载体

- 个人级工作门户
  任务看板 · 一键转 AI · 任务流回溯
- Z-AI-Workspace IDE 插件
  VSCode/Cursor 扩展 · HTTP 桥接 CLI 工具

编排调度

主控 Skill（Master Agent）：动态调度 · 上下文管理 · 分层递进式编排

需求分析 | 方案设计 | 编码实现 | 编译调试 | 测试验证 | 提交评审 | 总结提交

核心能力

- 需求分析 Skill
  AI 输出+人工确认
- 代码开发 Skill
  8 阶段递进编排
- 质量门禁 Skill
  扫描-修复闭环
- FT 测试 Skill
  自动执行·生成报告
- 代码评审 Skill
  三层约束体系
- 故障诊断 Skill
  端到端自诊断

工程底座

- git worktree 版本工程规范化
  任务独立工作区 · 并行开发 · 完成合入主线
- Skill 三层治理
  General 通用层 · Domain 领域层 · Components 组件层

Harness Engineering
驾驭工程闭环

约束 → 反馈 → 验证 → 自我修正

失败→自修复→重新验证
驾驭闭环贯穿架构每一层

两层工程底座 × 六大核心 Skill × 驾驭工程闭环，支撑「需求开发 + 运维诊断」两条一站式产线

一站式需求开发：全栈模式变革

✕ 传统模式：跨职能接力



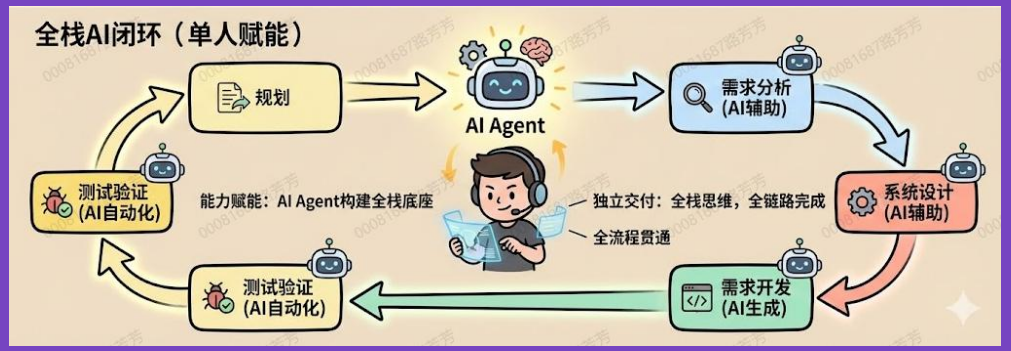
· 多角色（BA/SE/Dev/QA）串行协作

· 存在严重的信息熵增与感知衰减

· 角色边界固化，形成"知识孤岛"

· 交付物易偏离预期

✓ 全栈模式：单节点闭环



· **能力重构**：工程师转变为"研发指挥官"，利用 AI 独立完成全链路任务

· **认知一致**：消除跨角色传递损耗，从理解到实现知识一致

· **重心转移**：从"代码编写"转向"需求定义"与"结果确认"

解决思路 · 运维诊断

一站式运维诊断：打造"熄灯工厂"

基于 Cursor Agent 的代码理解与通识能力构建全代码工程，依托 Agent Skills 和 MCP 降低对人的依赖，从"人工主导、多人协作"演进为"**智能体驱动、自动闭环**"。



技术架构三层

01 Cursor Agent 智能诊断 Agent，故障分析、根因定位、修复建议

02 Agent Skills 日志分析、指标查询、链路追踪、知识检索

03 MCP 集成 对接监控平台、日志系统、告警中心、知识库

全流程自动化

故障发现→诊断分析→根因定位→修复执行→验证确认，全流程无需人工介入

智能决策引擎

基于历史数据和模式识别，AI 自主决策修复策略，降低人工判断依赖

7×24 小时值守

不受时间和人力限制，全天候监控和响应，确保系统稳定性

解决思路 · 核心理念

Harness Engineering（驾驭工程）：约束、反馈、验证、自我修正

设计和构建一套围绕 AI 模型的**约束、反馈、验证和修正系统**，确保 AI 智能体能够可靠、安全地生成和维护大规模生产级代码。



驾驭工程理念：工作中心转移

工作全部集中在构建 Harness 上：编写测试用例、定义架构规则、设计反馈循环，让 AI 在这套"驾驭系统"内不断迭代和自我修正。

工程师角色的两个转变

· 从"码农"变为"**架构师**"：不再亲自编写业务逻辑，而是设计 AI 运行的规则

· 从"实现者"变为"**评审者**"：定义什么是"正确"的代码，让系统自动拦截"错误"的代码

解决思路 · 技术体系

Agent Skills 能力支撑体系：分层递进式编排

主控 Skill（Master Agent）
动态调度与上下文管理

需求分析 | 方案设计 | 编码实现 | 编译调试 | 测试验证 | 提交评审 | 总结提交

标准化封装

以 Agent Skills 作为统一扩展标准，将领域知识与工作流沉淀为可编排、可复用的能力组件

资产化运营

Skills 以文件形式托管于代码仓库，确保知识资产的可维护、可管理与可继承性

高效化执行

分层资源加载与渐进式提示词披露机制，优化上下文窗口利用率，降低推理成本

工具底座：基于类 Cursor 工具构建，以 Agent Skills 为核心的能力支撑体系

03

实践情况

两层工程底座 + 六大核心 Skill + 三个真实案例

先修路再跑车：版本工程规范化 × Skill 三层治理

底座一　git worktree 版本工程规范化

每个任务单独创建一个 worktree，在独立工作区开发，互不影响，完成后再合入主线。

- **共享 .git 对象库**：新增 worktree 几乎不占额外磁盘
- **完全独立**：暂存区、未提交更改互不影响
- **无需反复 stash/checkout**：多任务并行
- **支持 IDE 多开**：不同 worktree 配置独立

为什么先做这个？ AI 并行执行多个任务的前提，是每个任务有干净、独立、可追溯的工程环境。

底座二　Skill 三层架构治理

General 部门通用	需求分析、方案设计、编译、FT 测试、代码静态分析、代码走查等通用能力；统一仓库，部门公共人力维护
Domain 领域通用	覆盖 vod、net、browse、dvb、factory_test、bsp、framework、decoder 等多个技术领域；领域人力维护
Components 组件定制	单个组件定制，跟随组件仓库；提供组件业务代码生成、组件用例设计与代码生成等定制化能力

通用层独立 git 仓库管理，定制层跟随组件仓库维护

实践情况 · 能力矩阵

六大核心 Skill：覆盖从需求到提交的全链路

01 需求分析 Skill

AI 输出 + 人工自然语言确认，明确"做什么、边界在哪里"，实现需求活动的左移

02 代码开发 Skill

方案设计→业务代码生成→用例代码生成→编译→静态检查→ZTS 测试→综合检查→人工审核→代码提交，8 阶段递进式编排

03 质量门禁 Skill

Lizard / Klocwork / Coverity 扫描工具剥离流水线，封装为独立子 Skill，扫描-修复闭环

04 FT 测试 Skill

测试剥离整体工作流，封装为独立 Skill；自动匹配用例模块，执行 ZTS 测试，生成报告

05 代码评审 Skill

通用红线 + 语言规则（Java/C++/JS/Python）+ 架构规则，三层约束体系保障 AI 代码生成质量

06 故障诊断 Skill

诊断前准备→经验初筛→日志分析定性→根因分析→修复决策与人工审核→代码修复，端到端自诊断自修复

实践情况 · 案例一

需求分析 Skill：AI 输出 + 人工确认，需求活动左移

目标

明确"做什么"、消除模糊、定义"边界在哪里"

动作

人机交互确认核心需求、挖掘隐含需求、明确边界条件与异常处理逻辑

输入

待分析的 MR 列表（基于 Skills + MCP 对接需求系统）

输出

清晰、无歧义的需求分析 MD、组件功能 MD

工作方式：人机协同的确认闭环

获取 MR 列表 → AI 分析与细化 → 人工自然语言确认 → 生成需求文档 → 进入开发

带来的改变：把需求工作流变为真正的"AI 输出 + 人工自然语言确认"，赋能人人可做需求分析，实现需求活动的左移

实践情况 · 案例二

质量门禁 Skill：编码-检查-修复的自动闭环

AI 编码
生成代码
→
触发扫描
Skill 执行
→
查看报告
问题定位
→
修复问题
即时修正
→
重新扫描
验证通过

扫描工具与流水线门禁对齐

Lizard 圈复杂度对齐流水线门禁阈值；Klocwork / Coverity 告警级别、缺陷分类定义保持与流水线规则一致，支持增量 / 全量两种扫描模式

差分分析：只关注新问题

对基线扫描报告和当前扫描报告进行智能差分，精准识别本次修改引入的新问题，过滤历史遗留问题

为什么要把扫描工具从流水线剥离？

流水线是"事后把关"，发现问题时代码已经写完；封装为 Skill 后，检查嵌入 AI 编码过程，**问题在生成阶段就被拦截和修复**——这就是"验证与自我修正"的落地。

实践情况 · 案例三

故障诊断 Skill：一个真实故障的自诊断全过程

端到端自诊断流程

诊断前准备：根据 Bug ID 获取故障描述、版本信息

故障经验初筛：历史经验匹配，辅助后续分析

日志 + 代码分析：问题定性（环境 / 软件 / 待定）

根因分析：日志 + 代码双向验证，输出诊断报告

修复决策：**置信度 > 70%** 自动进入代码修复，否则转人工审核

代码修复 → 验证确认 → 结束

真实案例：红灯闪烁频率异常的根因分析

三、根因分析

🎯 **核心问题**

红灯闪烁频率很快 = 多个LED闪烁线程并发运行 → 闪烁频率叠加

🔬 **技术根因**

层次	问题	证据	置信度
直接原因	多个MC_ThreadLedSparkle线程并发运行	日志显示持续46秒有MC_ThreadLedSparkle线程运行	100%
深层原因1	MC_LedSparkle函数缺少线程管理机制	代码行747：每次调用都创建新线程，没有停止旧线程	95%
深层原因2	防重复启动检查存在竞态条件	代码行572-581：检查和设置iPwrRedStatusNeed之间没有原子性保护	90%
触发条件	待机唤醒后短时间内多次调用	日志显示140ms内两次调用MC_LedSparkle	85%

系统给出的根因链（附证据与置信度）：

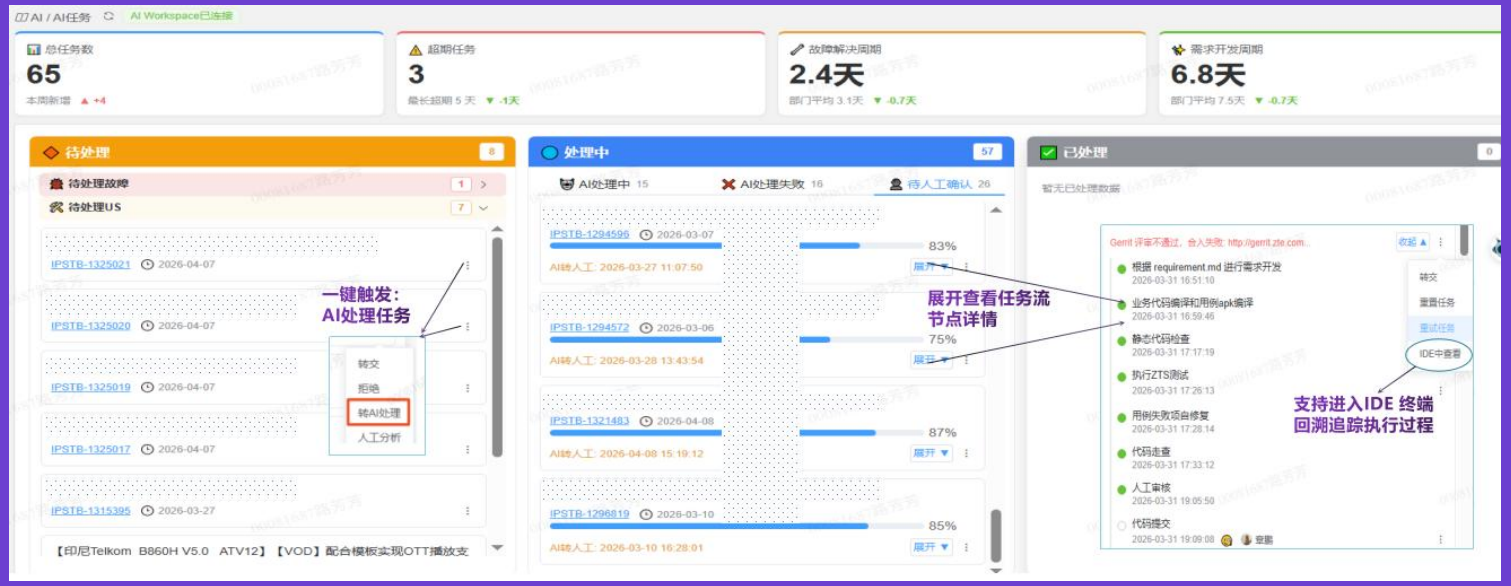
· **直接原因**：多个 MC_ThreadLedSparkle 线程并发运行 （100%）

· **深层原因**：函数缺少线程管理机制，每次调用都创建新线程 （95%）

· **触发条件**：待机唤醒后短时间内多次调用 （85%）

从"看日志猜原因"到"给证据链下结论"。

个人级工作门户 + IDE 插件：让 AI 任务"看得见、管得住"



个人工作看板：任务状态、执行流程、节点详情全程可视化

一键转 AI 处理

多任务并发、任务节点详情查看、一键重置与重试、支持夜间工程

Z-AI-Workspace 插件

基于 VSCode/Cursor 扩展，HTTP 桥接打通一体化平台与类 Cursor CLI 工具

任务流回溯

按会话 ID 进入 IDE 终端查看详情，与智能体继续交互、修正、恢复执行

AI 自动化降低人力成本，数据可视化提升决策效率，流程标准化保障工作质量——**人在关键环节确认，而不是在全程陪跑**

04 效果评价

不讲感受，只看数字

效果评价 · 落地收益

数字说话：两条产线的收益

一站式需求开发

400 单/月

一站式月均处理需求单，占整体交付需求 30%

≥ 6 倍

需求开发效率提升（简单需求 6 倍，复杂需求 8~12 倍）

≥ 70%

AI 生成代码采纳率（简单需求达 100%）

一站式故障诊断

200+ 单/月

一站式月均处理故障单，占整体故障 20%

70%+

故障分析结果平均采纳率

50%+

诊断结果平均采纳率

回顾：三件事，一条主线

①

模式变革

从多角色接力到单节点全栈闭环，消除人工转译产生的信息熵增

②

方法落地

Harness Engineering：约束、反馈、验证、自我修正，让 AI 可靠地产出生产级代码

③

体系支撑

工程底座 + Agent Skills 资产化运营，形成可复制的端到端提效范式

工程师的价值，正在从"写代码"转向"驾驭 AI"。

我们的工作，是让每一个工程师都成为 AI 的驾驭者。



2026 年 3-8 月一站式任务月度趋势


THANKS
谢谢聆听，欢迎交流